Problem 1

The town of Thirtyville is preparing for a large expansion. Naturally, the mayor of Thirtyville needs to decide how the electrical network for the town should be designed. Let $G = (V, E)$ be a connected undirected graph with positive edge weights $w : E \to \mathbb{R}^+$ representing the expanded town. The vertices in V represent locations and the edges in E represent potential power lines that can be built, where the weight $w(e)$ of an edge $e \in E$ is the cost in dollars to build that power line.
The set of vertices V is composed of two disjoint sets $H \subset V$ and $P \subset V; V = H \cup P$. The vertices in H represent houses, and the vertices in P represent where power plants will be built. Describe an algorithm to compute the cheapest subset of power lines $L \subseteq E$ to build so that every house $h \in H$ is reachable from a power plant $p \in P$ via the power lines in L. Note that houses do not need to have a direct power line to a power plant; a path composed of power lines suffices. Justify the correctness of your algorithm. A formal proof is not necessary. Analyze the runtime. [Hint: Reduce the problem to computing a minimum spanning tree in a different graph. Include all parts of a reduction in your answer.]

Solution: We interpret the solution as there could be one or multiple $p \in P$ for which there exists a cheapest power line which connects such $p \in P$ to some or all $h \in H$. Therefore, not all $p \in P$ should be used. This means the resulting graph need not to be connected and could be disjoint.

Algorithm: Following the hint, construct $G' = \{V', E'\}$, where s is a source vertex connected to all $p \in P$

$$V' = V \cup \{s\}$$

$$E' = E \cup \{(s, p) \mid p \in P\}$$

Let $w(s, p) = 0$ for all $p \in P$. See that it suffices to run a mst algorithm on G' and remove the edges connected to source node {s} to and compute the weights of the mst to obtain cheapest subset $L \subseteq E$.

Correctness: We first note by running the mst algorithm, all houses are connected to a at least one power plant $p \in P$ via direct connection or by a path. Once we remove the edges connected to {s}, we are left with a forest where each tree is connected to a power plant. By definition of minimum spanning tree, this forest gives us the cheapest subset of power lines $L \subseteq E$.

Runtime: Suppose we used Boruvka's algorithm. Then we have $O(E'logV') = O(ElogV)$. Constructing the graph takes $O(V'+E') = O(V+E)$. Therefore, our total tuntime is $O(ElogV)$.

Problem 2

While you were solving the previous problem, the mayor has realized building power plants at the previously considered locations will be far too costly. Instead, they must solve the following problem (with the same setup as before, for completeness): Let $G = (V, E)$ be a connected undirected graph with positive edge weights $w : E \to \mathbb{R}^+$ representing the expanded town. The vertices in V represent locations and the edges in E represent potential power lines that can be built, where the weight $w(e)$ of an edge $e \in E$ is the cost in dollars to build that power line.

Now, a power plant can be built at *any location in V* for a cost of *z dollars*, where $z > 0$ is a given parameter (in this problem, there is no partition of V into different kinds of vertices). The goal is now to select locations $P \subseteq V$ to build power plants *and* which power lines $L \subseteq E$ to build so that *every location in V* is reachable from a power plant $p \in P$ via the power lines in L. Furthermore, the total cost of the power plants and power lines must be minimized.

For example, a feasible solution with cost $z|V|$ is to set $P = V$ and $L = \emptyset$, meaning that a power plant is built at every location, which is valid because every location is trivially reachable from itself with a power plant. For another example, a feasible solution is to build a single power plant (somewhere) and decide which edges to connect all locations it. In between these extremes is the possibility to build more than one but less than $|V|$ power plants.

Describe an algorithm for this problem. Justify the correctness of your algorithm. A formal proof is not necessary. Analyze the runtime. [Hint: Reduce the problem to computing a minimum spanning tree in a different graph (perhaps similar, but not the same, as that for the previous problem). Include all parts of a reduction in your answer.]

Solution: We observe that we should only substitute v for a power plant if z provides us a minimum connection hence:
Algorithm: Following the hint, construct $G' = \{V', E'\}$,

$$V' = V \cup \{s\}$$

$$E' = E \cup \{(s, v) \mid v \in V\}$$

Let $w(s, v) = z$ for all $v \in V$. See that it suffices to run a mst algorithm on G' to obtain tree T. Let $P = \{v \in V \mid (s, v) \in T\}$ and $L = \{(u, v) \in E \mid (u, v) \in T\}$. {s} to and compute the weights of $L \subseteq E$ to obtain cheapest cost.

Correctness: We first note by running the mst algorithm, all vertexes are either a power plant or connected to a power plant through a path in the MST. And by definition of minimum spanning tree, this forest gives us the cheapest subset of power lines $L \subseteq E$, while also optimizing for power plant cost z.

Runtime: Suppose we used Boruvka's algorithm. Then we have $O(E'logV') = O((E + V)logV)$. Constructing the graph takes $O(V' + E') = O(V + E)$. Therefore, our total runtime is $O(ElogV)$, since worse case $|E| \geq |V| - 1$.

Problem 3

Phoebe is an artisan potter that specializes in terracotta. She offers a vast catalog of vases with various sizes, colors, and shapes, and she has just received an immensely large and complicated order for n different designs of vases, $V_1, V_2, \ldots, V_n$. Each design V_i takes a distinct number of minutes, s_i, for Phoebe to first sculpt by hand, and then another distinct number of minutes, f_i, for her to finish by firing it in her kiln. For purposes of this problem, suppose her kiln can fit any number of sculpted vases to fire and that it is OK for fully-fired vases to remain in the kiln until all vases are done firing.

Phoebe can only sculpt one design at a time, but immediately after she finishes sculpting a vase she adds it to the others in the kiln (if any) so that she can begin sculpting another vase. Once all vases are sculpted and fired in the kiln for at least as long as they all require, she pulls them all out at once and her job is done. Your goal is to help Phoebe determine which order she should sculpt the n designs in order to minimize the time when her job is done.

(a) Prove that ordering the designs in increasing order of s_i is not optimal by providing a counterexample by providing an input for which the ordering does not yield the best possible end time.

(b) Same as the last part, but with ordering in increasing order of $s_i + f_i$.

(c) State an ordering for Phoebe to use and prove that it is optimal. You may find the following proof template useful[2]:

(i) Compare your ordering to that of an arbitrary optimal ordering: If the orderings are not identical, then there are some pairs of designs in the optimal ordering that are out-of-order with respect to your ordering.

(ii) Consider the optimal ordering *with the fewest such pairs*.

(iii) Show that swapping two adjacent out-of-order designs in that ordering reduces the number of out-of-order pairs without increasing the overall end time.

(iv) Conclude that, because the resulting order has fewer out-of-order pairs and optimal end time, your ordering is indeed optimal by contradiction.

Solution:
(a) Suppose we have V_1 with $s_1 = 1$, $f_1 = 1$ and V_2 with $s_2 = 2$, $f_2 = 2$. According to the ordering rule, Phoebe should sculpt V_1 before sculpting V_2, which takes 5 minutes. However, if she started with V_2, total time would be 4 minutes. Thus it is not an optimal ordering.

(b) Suppose we have V_1 with $s_1 = 1$, $f_1 = 1$ and V_2 with $s_2 = 2$, $f_2 = 2$. According to the ordering rule, Phoebe should sculpt V_1 before sculpting V_2, which takes 5 minutes. However, if she started with V_2, total time would be 4 minutes. Thus it is not an optimal ordering.

(c) Ordering the designs in decreasing order of f_i is optimal.
Proof:
Suppose there exists some optimal ordering O that is different from our hypothesized optimal order H. In O, there must be some pairs of adjacent vases that are out of order with respect to H. Assume there is only one out-of-order pair (V_i, V_{i+1}) in O where $f_i < f_{i+1}$. Swap V_i and V_{i+1} to form a new ordering O'. We need to show that this swap does not increase the overall completion time.
We first define the total sculpting time up to V_i as:

$$S_i = \sum_{k=1}^{i} s_k$$

Before the swap, the total completion time considering all vases from V_1 to V_n is:

$$T_{\text{old}} = \max_{1 \le k \le n} (S_k + f_k)$$

After swapping V_i and V_{i+1}, the new maximum completion time is:

$$T_{\text{new}} = \max_{1 \le k \le n} \left(S'_k + f'_k\right)$$

where S'_k and f'_k represent the new sculpting and firing times after the swap.
Swapping V_i and V_{i+1} only affects the *i*th and *(i+1)*th item in T. According to our definition, we have:

$$S_{i+1} + f_{i+1} > S'_i + f_{i+1}$$

and

$$S_{i+1} + f_{i+1} > S'_{i+1} + f_i$$

This guarantees that the *i*th and *(i+1)*th item in T will decrease after swapping, ensuring $T_{\text{new}} \le T_{\text{old}}$.

Problem 4

Let X be a set of n intervals on the real line labeled $1, \ldots, n$. We say that a subset of intervals $Y \subseteq X$ covers X if the union of all intervals in Y is equal to the union of all intervals in X. The size of a cover is just the number of intervals.
Describe a greedy $O(n \log n)$-time algorithm to compute the smallest cover of X. Assume that your input consists of two arrays $L[1..n]$ and $R[1..n]$, where $(L[i], R[i])$ is the i-th interval in X. Note that X as given is ordered arbitrarily. For simplicity, assume that no two endpoints of intervals are equal; that is, all elements of L, R are distinct. Prove the correctness of the algorithm (including an exchange argument) and analyze its runtime complexity.

Solution:

Algorithm:
Step 1: Sort the intervals in an increasing order of $L[i]$.
Step 2: Include the left most interval i into Y.
Step 3: Consider the subset of remaining intervals $M = \{j \mid L[i] < L[j] < R[i] \ \ \& \ \ R[i] < R[j]\}$. Sort M in an increasing order of $R[j]$. Include the interval j of the largest $R[j]$.
Step 4: Repeat Step 3 until the subset M is empty. Return Y.

Proof of correctness:
Let Y^* be an optimal solution different from our greedy solution Y. Assume Y^* has the smallest number of intervals but is not identical to Y. Y and Y^* are both sorted in increasing order of left end of the intervals. Let j be the first index where Y and Y^* differ. This means the intervals selected by Y and Y^* up to $j-1$ are the same, but the j-th interval is different. Let I_g be the j-th interval in Y, and let I_o be the j-th interval in Y^*. Since I_g was chosen by the greedy algorithm, I_g must end no earlier than I_o. Replace I_o in Y^* with I_g. This new set $Y^{*'}$ still covers all the intervals in X, and has the same number of intervals as Y^*, ensuring the greedy solution is not worse than optimal solution.

Runtime complexity: Sorting the intervals takes less than or equal to $O(log(n))$ time. In worst case where all intervals are included in Y, we sort n times. Thus total runtime is $O(nlog(n))$.